From Design Science Research Theory to Repository-Native Operationalization

A White Paper for the DSR Framework

David Ochabski

2026-05-18

Contents

Version note

This release is prepared as an open artifact white paper. It is designed to be readable as a normal human-facing paper and also structured enough for repository automation, AI ingestion, archival deposit, and future validation work.

Original project-authored white paper content and metadata are intended for CC0-1.0 dedication, matching the repository's maximally open non-code licensing policy. Citation is appreciated for scholarly traceability, but it is not a license condition for CC0 content.

The paper should not claim independent external validation, semantic consensus, or L5 archival/publication-ready status until those events are explicitly recorded in the DSR Framework repository.

Abstract

Design Science Research (DSR) is an artifact-centered research tradition in which researchers identify a relevant problem, design and instantiate an artifact or design entity, evaluate that artifact against explicit objectives and criteria, and communicate reusable or projectable design knowledge (March and Smith 1995; Hevner et al. 2004, 2024; Peffers et al. 2007; Gregor and Hevner 2013; vom Brocke, Hevner, and Maedche 2020). The challenge addressed by this white paper is operational: DSR theory provides rich guidance, but DSR work is often documented in prose that obscures artifact boundaries, problem-to-requirement traces, evaluation alignment, contribution claims, and reuse conditions (vom Brocke and Maedche 2019; Weigand, Johannesson, and Andersson 2021; Hevner et al. 2024). This paper synthesizes the DSR theory base and explains how the DSR Framework repository operationalizes it as a GitHub-native artifact package (Ochabski 2026a, 2026b). The framework treats DSR documentation as a structured, traceable, validation-ready system that includes models, vocabularies, schemas, templates, checklists, records, crosswalks, rubrics, examples, and nested artifacts. The paper positions the repository as a compound DSR artifact: a framework for producing, reviewing, releasing, citing, and reusing DSR artifact packages. It also proposes a publication and dissemination strategy that preserves a single canonical source of truth while increasing human and machine discoverability through GitHub, Zenodo, OSF, Octopus, and selected profile/dissemination channels.

Keywords

Design Science Research; design knowledge; artifact ontology; DSR evaluation; DSR transparency; repository-native research; research artifact packaging; GitHub; Zenodo; Open Science Framework; Octopus; FAIR; AI-legible documentation.

Recommended citation

Ochabski, D. (2026). *From Design Science Research Theory to Repository-Native Operationalization: A White Paper for the DSR Framework* (Version 1.0.0). DSR Framework. https://doi.org/10.5281/zenodo.20264299

Executive summary

This white paper is framed as a **repository-native methodological white paper with an open artifact package**. It is not mainly a policy brief, a promotional white paper, a dataset descriptor, or a conventional literature review. Its contribution is partly conceptual, partly methodological, and partly infrastructural: it explains what DSR is, how DSR can be operationalized, and how the DSR Framework repository is operationalizing it.

The central thesis is:

> Design Science Research becomes more reviewable, reusable, citable, and AI-legible when its core commitments are represented as structured repository artifacts: problem and context records, artifact specifications, requirements traces, design-rationale records, evaluation plans and reports, contribution claims, controlled vocabularies, schemas, validation shapes, rubrics, examples, release records, and preservation metadata.

The paper makes three linked arguments.

First, DSR is not merely "building something." It is artifact-centered inquiry that connects a practically significant problem, a designed artifact or design entity, rigorous grounding in prior knowledge, demonstration and evaluation, and reusable design knowledge (March and Smith 1995; Hevner et al. 2004; Peffers et al. 2007; Johannesson and Perjons 2021).

Second, DSR needs operational structure. A complete DSR record should capture the problem instance, problem class, context, stakeholders, input knowledge, objectives, requirements, artifact/design entity, build trace, demonstration/use trace, evaluation evidence, contribution claim, design knowledge, boundary conditions, and transparency trace (vom Brocke and Maedche 2019; Maedche et al. 2019; Weigand, Johannesson, and Andersson 2021; Hevner et al. 2024).

Third, the DSR Framework repository is a credible attempt to implement that structure. Its strongest contribution is traceability discipline: it treats GitHub not simply as a host but as a research artifact stewardship environment where files, versions, metadata, validation records, release records, and crosswalks carry methodological evidence (Ochabski 2026a, 2026b).

The recommended publication architecture is intentionally conservative: use GitHub as the canonical living source, Zenodo as the DOI-bearing archival release, OSF as a project/discovery hub or optional preprint layer, and Octopus as an optional research-process record once the paper has stable modular sections. Use ResearchGate, Academia.edu, LinkedIn, Substack, and similar services for dissemination only. Use Mendeley Data or Dryad only for genuine data packages, not as the canonical home for the white paper itself.

---

1 Introduction

Design Science Research has an operational problem beneath its theoretical richness. The field gives researchers a strong paradigm: identify relevant problems, design artifacts, evaluate them, and contribute design knowledge (Hevner et al. 2004; Peffers et al. 2007; Gregor and Hevner 2013; vom Brocke, Hevner, and Maedche 2020). Yet many DSR outputs remain hard to inspect because their most important relations are buried in narrative. A reader may see an artifact but not the problem class it addresses, an evaluation

but not the claim it supports, or a contribution statement but not the boundary conditions that make the claim reusable rather than overgeneralized (Maedche et al. 2019; Baskerville et al. 2018; Hevner et al. 2024).

This paper treats DSR documentation as a design problem in its own right. If DSR is artifact-centered inquiry, then a framework for documenting DSR artifacts can itself be designed, evaluated, reviewed, released, and improved as a DSR artifact. The DSR Framework repository follows that logic by translating DSR theory into structured repository products: models, vocabularies, schemas, templates, rubrics, records, checklists, examples, and crosswalks (Ochabski 2026a).

The paper distinguishes two layers that should not be conflated.

1. **DSR theory**: the scholarly and professional literature on artifact-centered research, problem relevance, rigorous grounding, build/evaluate logic, contribution, evaluation, transparency, reliability, replication, projectability, and publication.
2. **Repository operationalization**: the specific implementation of those ideas in the DSR Framework repository and its related Documentation Protocol repository (Ochabski 2026b).

The purpose is not to replace the DSR literature with a single repository template. The purpose is narrower and more practical: to show how DSR commitments can be represented in a source-grounded, reviewable, reusable, citable, and machine-actionable artifact package.

1.1 Audience

The primary audiences are DSR researchers, doctoral supervisors, methodology instructors, reviewers, editors, repository maintainers, and AI-assisted research infrastructure builders. Practitioners may also use the framework where DSR artifacts must be documented for reuse, review, audit, or publication.

1.2 White paper type

The recommended genre is a repository-native methodological white paper with an open artifact package. A conventional review article would understate the designed artifact package. A dataset deposit would overemphasize the source extractions. A policy brief would be too thin. A software paper would be too narrow. The right genre is a white paper that explains the theory, states the operational model, and points to a versioned repository artifact.

---

2 What Design Science Research Is

Design Science Research is an artifact-centered research paradigm. Its central operation is not observation alone and not implementation alone, but the disciplined transformation of a relevant problem into an evaluated artifact and codified design knowledge (Simon 1996; March and Smith 1995; Hevner et al. 2004). A DSR study therefore needs a problem, context, input knowledge, objectives, artifact, instantiation or demonstration, evaluation evidence, contribution claim, and transparency trace (vom Brocke and Maedche 2019; Hevner et al. 2024).

A compact formalization is useful:

```
DSR = <P, C, Rq, K_in, O, A, I, E, DK, K_out, T>
```

where `P` is the problem, `C` the context, `Rq` the research question, `K_in` the input knowledge, `O` the objectives and requirements, `A` the artifact or design entity, `I` the instantiation or use case, `E` the evaluation evidence, `DK` the generated design knowledge, `K_out` the output knowledge, and `T` the transparency trace.

Table 1. Consensus commitments in DSR. This synthesis draws from foundational and recent DSR methodology, contribution, evaluation, artifact-ontology, and transparency sources (March and Smith 1995; Hevner et al. 2004, 2024; Peffers et al. 2007; Gregor and Hevner 2013; Venable, Pries-Heje, and Baskerville

2016; Baskerville et al. 2018; Weber 2018; vom Brocke, Hevner, and Maedche 2020; Johannesson and Perjons 2021; Weigand, Johannesson, and Andersson 2021).

Commitment	Operational meaning	Failure avoided
Artifact centrality	A designed artifact or design entity is the object of inquiry.	Pure explanation or critique without design contribution.
Practical problem relevance	The artifact is justified by a meaningful problem in context.	Abstract artifact construction without stakeholder or problem grounding.
Rigor through knowledge base	The design and evaluation draw from prior theory, artifacts, methods, patterns, and evidence.	Unjustified invention.
Build/evaluate logic	Design and evaluation are coupled and often iterative.	Undemonstrated or unevaluated artifact claims.
Design knowledge contribution	The study contributes reusable or projectable knowledge, not just local delivery.	Routine implementation overclaim.
Evaluation validity	Evaluation fits the artifact, maturity, context, criteria, and claim type.	Method-claim mismatch.
Communication	The study communicates problem, artifact, method, evaluation, contribution, and applicability.	Artifact delivery without research communication.
Accumulation	Knowledge is codified so later researchers can reuse, adapt, replicate, or challenge it.	Isolated one-off artifact reports.
Genre plurality	DSR allows multiple artifact and contribution genres.	False requirement that all DSR must look like one paper template.

This interpretation sets the boundary between DSR and ordinary consulting or software delivery. A local implementation may be useful, but a DSR contribution requires explicit design knowledge, evaluation evidence, and boundary conditions (Gregor and Hevner 2013; Baskerville et al. 2018; Akoka et al. 2023).

---

3 How DSR Can Be Operationalized

The operational problem is to convert DSR commitments into a record architecture that can be reviewed, validated, reused, and cited. This paper proposes a minimum DSR record architecture with twelve required modules. The architecture integrates the DSR Grid, problem-space conceptualization, artifact ontology, evaluation alignment, contribution theory, transparency guidance, and reliability/replication concerns (vom Brocke and Maedche 2019; Maedche et al. 2019; Weigand, Johannesson, and Andersson 2021; Venable, Pries-Heje, and Baskerville 2016; Hevner et al. 2024; Storey, Baskerville, and Kaul 2025).

Table 2. Minimum DSR record architecture.

Module	Purpose	Minimum contents
Source and project identity	Establish provenance and reviewability.	title, authors, version, source role, DSR genre, extraction confidence

Module	Purpose	Minimum contents
Problem space	Prevent weak problem grounding.	problem instance, problem class, severity, relevance, solvability
Context	Bound validity and projectability.	organization, domain, stakeholders, constraints, artifact network, boundary conditions
Input knowledge / solution space	Establish rigor grounding.	kernel theories, prior artifacts, methods, design patterns, practitioner knowledge
Objectives and requirements	Translate the problem into evaluable design targets.	objectives, meta-requirements, design requirements, acceptance criteria
Artifact/design entity	Define the designed object of inquiry.	artifact type, artifact universal, instantiation, specification, make plan, use plan, capacity specification
Build and design rationale	Avoid artifact black boxing.	design decisions, alternatives, rationale, build trace, iteration trace
Demonstration and use	Show feasibility and situated use.	scenario, prototype, pilot, use case, implementation context, user role
Evaluation	Support utility and claim validity.	evaluation object, criteria, timing, setting, evidence mode, stakeholders, findings, limitations
Design knowledge and contribution	Distinguish research from routine design.	contribution object, knowledge goal, scope, novelty, evidence basis, boundary conditions
Reliability, replication, accumulation	Support cumulative DSR.	reliability target, replication type, reuse conditions, projectability statement
Transparency and responsible disclosure	Support trust without overload.	process, problem-space, solution-space, build, evaluation, and contribution transparency

This architecture turns DSR into a linked system rather than a linear narrative. The key spaces are problem space, context space, solution space, design space, artifact space, evaluation space, knowledge space, and communication/transparency space. The most important edges are:

```
problem -> objective -> requirement -> design decision -> artifact -> demonstration -> evaluation -> co
```

Operationalization should not over-formalize every judgment. Some checks can be automated, such as whether a study has a problem class, artifact, evaluation plan, and contribution claim. Other checks require human review, such as whether the problem is important, whether the artifact is novel enough, whether the evidence adequately supports the claim, and whether boundary conditions are credible (Hevner et al. 2024; Brendel et al. 2021; Storey, Baskerville, and Kaul 2025).

3.1 Evaluation alignment

A DSR evaluation record should distinguish timing, function, setting, object, evidence mode, criterion, stakeholder, and claim supported. Demonstration is not evaluation unless criteria, evidence, and claims are defined. Evaluation evidence should be represented separately from contribution claims: evaluation says whether and under what conditions the artifact works; contribution explains what reusable knowledge has been added (Venable, Pries-Heje, and Baskerville 2016; Johannesson and Perjons 2021; Hevner et al. 2024).

3.2 Contribution alignment

A strong design-knowledge claim links context, intervention, mechanism or rationale, outcome, and boundary condition. This makes the contribution portable enough to be useful while still bounded enough to avoid overgeneralization (Baskerville et al. 2018; Akoka et al. 2023; Reining et al. 2022).

---

4 Repository-native operationalization

The DSR Framework repository operationalizes DSR by making methodological evidence structural. Instead of asking authors only to write a narrative, it gives them repository locations and controlled information items for the claims a DSR study needs to support. This repository-native design aligns with DSR calls for artifact traceability, design knowledge codification, transparency, and accumulation (vom Brocke and Maedche 2019; Dickhaut, Janson, and Leimeister 2022; Hevner et al. 2024; Reining et al. 2022).

Table 3. Repository-native design principles.

Design principle	Repository expression
Source of truth discipline	Controlled YAML and metadata files capture the authoritative structured record.
Human legibility	Markdown documentation explains the structured files for researchers, reviewers, and maintainers.
AI legibility	Stable identifiers, schemas, controlled vocabularies, and file maps support machine ingestion and automated review.
Non-overclaiming	Conformance levels and release records distinguish draft, documented, reviewable, exercisable, reusable, and archival/publication-ready status.
Traceability	Crosswalks and records connect source basis, problem framing, design choices, evaluation evidence, contribution claims, and release decisions.

The repository is therefore not just a static guide. It is a compound artifact package containing specifications, models, vocabularies, schemas, templates, checklists, rubrics, records, crosswalks, examples, documentation, metadata, release controls, and nested artifacts (Ochabski 2026a).

4.1 The role of GitHub

GitHub is useful here because the artifact is file-based and versioned. A repository can retain commits, releases, tags, issue discussions, pull requests, review records, validation scripts, package inventories, and citation metadata. That does not make GitHub an archive by itself. It makes GitHub the living source-of-truth environment that should be paired with an archival DOI-bearing release repository. This distinction is consistent with the broader principle that research software and research artifacts need both working repositories and citable release/preservation metadata (FAIR for Research Software Working Group 2022; Association for Computing Machinery 2020).

4.2 The role of the Documentation Protocol

The Documentation Protocol supplies the meta-protocol: source-of-truth separation, package conformance, review records, release controls, preservation logic, citation metadata, licensing, and service integration. The DSR Framework applies that meta-protocol to the DSR domain. This relationship should be stated explicitly so readers understand why the framework is packaged as a repository artifact rather than only as a prose methodology paper (Ochabski 2026b).

5 How we are operationalizing DSR through the repositories

The current operationalization can be described as a three-layer artifact system. The layers should be presented as an evolving DSR artifact package, not as final evidence of field-wide validation.

5.1 Layer 1: The DSR Framework repository

The root DSR Framework repository defines the general framework for planning, documenting, evaluating, reviewing, and publishing DSR artifacts. It includes package-control files, metadata, artifact profiles, conformance declarations, specs, models, vocabularies, schemas, templates, checklists, rubrics, records, crosswalks, examples, and documentation (Ochabski 2026a).

Its core claim should remain bounded: it is a reusable-stable DSR documentation and review framework. It should not yet claim independent external validation, universal disciplinary standard status, or automatic validation of downstream DSR projects.

5.2 Layer 2: The DSR theory operational kernel

The nested DSR theory operational kernel turns the DSR theory synthesis into machine-readable and reviewable materials: concept inventory, competency questions, ontology, SKOS vocabulary, SHACL shapes, JSON Schema, YAML templates, crosswalks, source-basis records, and validation records. This layer operationalizes DSR concepts such as artifact universal, artifact instantiation, problem class, evaluation evidence, contribution claim, transparency trace, reliability, replication, and anti-patterns (Weigand, Johannesson, and Andersson 2021; Hevner et al. 2024; Storey, Baskerville, and Kaul 2025).

This is the most direct example of DSR theory becoming operational: it translates concepts into controlled terms, file structures, validation targets, and review questions.

5.3 Layer 3: White paper and release package

This white paper remains a bounded artifact package under:

`artifacts/dsr-framework-white-paper/`

Recommended retained files include:

```
README.md
whitepaper/chapters/*.md
whitepaper/whitepaper_unified.md
whitepaper/references.bib
metadata/CITATION.cff
metadata/zenodo-json-template.json
metadata/service-integration-profile.yaml
records/decisions/record-decision-0001-whitepaper-type-and-platforms.yaml
records/reviews/record-whitepaper-citation-and-build-audit.md
records/releases/record-release-checklist-whitepaper.yaml
repo_integration/repo-update-checklist.md
```

This v1.0.0 white paper release preserves the same non-overclaiming discipline as the root framework: it is a citable methodological white paper and artifact package, not a claim of independent external validation, semantic consensus, or L5 archival/publication-ready status.

6 Publication and dissemination strategy

The publication strategy should maximize visibility without creating multiple competing sources of truth. The recommended architecture is:

```
Canonical living source: GitHub
Archival citation snapshot: Zenodo DOI release
Project/discovery hub: OSF project, optional OSF Preprint
Research-process publication: Octopus, optional after stable modularization
Data-only secondary deposits: Mendeley Data or Dryad only if releasing a genuine data package
Profile/dissemination mirrors: ResearchGate, Academia.edu, LinkedIn, Substack, ORCID, personal website
Teaching/OER derivative: OER Commons or VIVA Open only after adapting the paper into learning materials
```

This strategy follows a repository-artifact logic rather than a scatter-and-mirror logic. The living source, archived release, and dissemination links should have distinct roles.

6.1 Recommended platform roles

Platform	Recommended role	Use now?	Rationale
GitHub	Canonical living source and source-of-truth repository.	Yes	Best fit for Markdown, metadata, version control, issue review, release records, schemas, validation scripts, and repo-native DSR logic.
Zenodo	DOI-bearing archival snapshot of GitHub release.	Yes	Best fit for citable release snapshot and long-term scholarly reference.
OSF	Project/discovery hub; optional preprint and linked materials.	Yes, after GitHub release and Zenodo DOI are available	Useful for open-science visibility, supplemental files, and discovery. Keep GitHub/Zenodo canonical.
Octopus	Modular research-process record.	Optional	Useful if you want to publish problem, method, results/synthesis, interpretation, and application as linked units. Do after the paper is stable.
SSRN	Discipline-facing preprint for IS/management/social-science audiences.	Optional	Consider if you want social-science and management discovery; do not make it canonical.
Mendeley Data	Data package only.	Optional later	Use only for source extraction corpus, concept inventory, or artifact data that can be openly licensed and safely shared.

Platform	Recommended role	Use now?	Rationale
Dryad	Curated data package only.	Usually no for this paper	Not the right primary home for a white paper; use only if releasing reusable research data under compatible terms.
ResearchGate	Profile dissemination link or licensed full-text mirror.	Optional	Good for visibility, not canonicality. Link to GitHub/Zenodo.
Academia.edu	Profile dissemination link or licensed full-text mirror.	Optional	Good for visibility, not canonicality. Link to GitHub/Zenodo.
OER Commons / VIVA Open	Teaching derivative or workbook.	Not yet	Use after making an instructional version, not for the primary white paper.
arXiv	Preprint only if a suitable category and format fit.	Probably no	DSR framework/methodology may fit better on OSF Preprints or SSRN unless there is a clear computing/information-science framing.
bioRxiv / medRxiv	Not relevant.	No	The paper is not a life-science or medical preprint.
HAL	Optional institutional/open archive mirror.	Optional	Consider only if you want a European open-archive mirror and can maintain metadata consistency.

The main rule is simple: do not scatter the white paper as unrelated uploads. Create one canonical GitHub release, archive it through Zenodo, then point all other platforms to that release or deposit a clearly labeled derivative.

---

7 Limitations, evaluation, and roadmap

The DSR Framework should be presented favorably but bounded. The available materials support a strong claim about conceptual synthesis, repository design, traceability, and self-application. They do not yet support a strong claim about external adoption, independent semantic adequacy, downstream empirical utility, or field-wide consensus.

7.1 Current strengths

1. The framework preserves DSR's identity boundary: artifact-centered inquiry, problem relevance, rigorous grounding, evaluation, and design knowledge (Hevner et al. 2004; Peffers et al. 2007; Gregor and Hevner 2013).
2. It operationalizes traceability instead of merely recommending it (vom Brocke and Maedche 2019; Hevner et al. 2024).

3. It distinguishes demonstration from evaluation and artifact utility from knowledge contribution (Venable, Pries-Heje, and Baskerville 2016; Baskerville et al. 2018).
4. It includes a theory operational kernel with ontology, SKOS, SHACL, schemas, competency questions, and concept inventory artifacts (Ochabski 2026a).
5. It uses conformance and release controls to avoid premature L5 claims (Ochabski 2026b).

7.2 Current limitations

1. The framework has not yet been independently validated by external DSR experts.
2. The ontology and controlled vocabulary need semantic review before they should be treated as stable disciplinary representations.
3. Repository self-application supports coherence, not general empirical utility.
4. The source extraction corpus and generated summaries require final human review before publication.
5. Platform release work requires manual authentication and metadata checks.

7.3 Recommended evaluation roadmap

Stage	Evaluation action	Evidence retained
Internal review	Check consistency among white paper, repo status, artifact-profile, package inventory, and release records.	Review record and issue log.
Build validation	Render Markdown to HTML/PDF and check links, headings, tables, citations, and accessibility.	Build log and release checklist.
Source review	Verify all core literature references, DOIs, and claims against source records.	Citation audit record.
Expert review	Ask 2-3 DSR scholars or advanced reviewers to assess theory fit and overclaiming.	External review records and response matrix.
Downstream pilot	Use the framework in one or more DSR studies or teaching contexts.	Pilot package, evaluation report, revised rubrics.
Archival release	Freeze a release, mint DOI, preserve metadata, and update citation records.	GitHub release, Zenodo DOI, metadata freeze record.

The roadmap should be published as a plan, not as evidence already completed. Reliability, replication, and transparency concerns should remain part of the evaluation agenda rather than being treated as solved by repository packaging alone (Brendel et al. 2021; Storey, Baskerville, and Kaul 2025; Hevner et al. 2024).

---

8 Conclusion

Design Science Research needs more than persuasive prose. It needs a way to preserve the relations among problem, context, requirements, artifact, design rationale, evaluation, contribution, boundary conditions, and reuse. The DSR Framework addresses that need by operationalizing DSR as a repository-native artifact package.

The contribution is strongest when stated modestly and precisely. The framework does not solve all of DSR and does not replace scholarly judgment. It makes DSR claims more explicit, inspectable, reusable, citable,

and challengeable. It gives researchers and reviewers a structured way to ask: What is the problem? What is the artifact? What knowledge grounds it? What was built? What was demonstrated? What was evaluated? What evidence supports the contribution? Where does the claim apply? What remains unknown?

The recommended next step is to release this white paper as a versioned GitHub artifact package, archive the release through Zenodo, link it through OSF, and use secondary services only as discovery or derivative channels. That path protects canonicality while making the work visible to both human readers and AI/repository systems.

Appendix A: Source basis and provenance

This white paper was prepared from three primary review/synthesis files in the Dropbox `DSR Theory Synthesis` folder:

1. `Design science research theory synthesis.md` - theory synthesis, consensus commitments, artifact theory, process theory, evaluation theory, validity/reliability/replication, contribution theory, transparency, and propositions.
2. `operational_synthesis_dsr.md` - operational DSR definition, minimum DSR record architecture, system ontology, artifact operationalization, process model, evaluation model, validation rules, and anti-pattern controls.
3. `dsr_framework_narrative_review.md` - narrative review of the DSR Framework repository and Documentation Protocol, with emphasis on repository-native operationalization and the `artifacts/` directory.

The package also includes supporting structured materials copied into `sources/`, including the concept inventory, competency questions, SKOS vocabulary, unified extraction sidecar, unified extraction chunks, rubrics, and wicked-problem workflow.

Source-use rule

The white paper should cite the review/synthesis documents as working synthesis materials only when needed. For publication, core DSR claims should cite the underlying literature listed in the references rather than treating the generated synthesis files as independent scholarly authorities.

Copyright and publication caution

Do not publish copyrighted source PDFs or extraction records containing excessive verbatim source text unless rights have been checked. Publish derived metadata, citation records, concept inventories, and original synthesis text only after human review.

Appendix B: Platform matrix

Platform	Canonical?	Best object to post	Recommended timing	Notes
GitHub	Yes	Markdown source, build files, metadata, release records	First	Use as living source of truth.

Platform	Canonical?	Best object to post	Recommended timing	Notes
Zenodo	Yes for archival snapshot	GitHub release archive, PDF, unified Markdown, metadata	After GitHub release	Use DOI-bearing snapshot.
OSF	No	Project page, optional preprint, links to GitHub/Zenodo, supplemental files	After GitHub release and Zenodo DOI	Use for discovery and open-science hub.
Octopus	No	Modular research-process records	After stable paper	Publish problem/method/results/interpretation chain if useful.
SSRN	No	Preprint PDF	Optional	Use if you want social-science/management/IS preprint discovery.
Mendeley Data	No	Data package, not paper	Optional later	Use only for data/materials that are safe and legal to share.
Dryad	No	Curated data package, not paper	Usually not needed	Consider only for reusable data requiring data repository curation.
ResearchGate	No	Link or licensed full-text mirror	After DOI	Use for visibility.
Academia.edu	No	Link or licensed full-text mirror	After DOI	Use for visibility.
OER Commons / VIVA Open	No	Teaching derivative, workbook, module	Later	Create after adapting for instruction.
arXiv	No	Preprint if category fit is clear	Optional, probably not first	Use only if scope fits.
bioRxiv / medRxiv	No	None	Never for this paper	Not relevant.
HAL	No	Archive mirror or preprint	Optional	Consider only with metadata consistency plan.

Decision: keep GitHub + Zenodo as the canonical pair. Everything else should point back to that pair or be clearly labeled as a derivative.

Appendix C: Codex build plan

This package is optimized so Codex can finish the release with minimal token use.

Files Codex should read first

1. `README.md`
2. `codex_handoff.md`
3. `whitepaper/whitepaper_unified.md`
4. `whitepaper/references.bib`
5. `publication_strategy.md`
6. `repo_integration/repo-update-checklist.md`

Files Codex should avoid reading unless necessary

1. `sources/dsr-source-operational-extractions-unified-part001.yaml`
2. `sources/dsr-source-operational-extractions-unified-part002.yaml`
3. Any full extraction YAML unless revising source registry appendices.

First Codex task

Apply the citation/build patch, run the Markdown assembly script, render HTML and PDF if Pandoc/LaTeX are available, and update only the necessary repository index files. Do not tag, create a GitHub release, publish to Zenodo, or publish externally until final human edit and metadata freeze.

Build commands

```
python scripts/assemble_whitepaper.py
make html
make pdf
python scripts/check_whitepaper_build.py
```

If LaTeX is not available, skip PDF and produce HTML plus unified Markdown.

Appendix D: Repository update checklist

- Create branch: `whitepaper/citation-build-fix`.
- Preserve the package under `artifacts/dsr-framework-white-paper/`.
- Update chapter source files, not only `whitepaper/whitepaper_unified.md`, because the assembly script regenerates the unified manuscript.
- Replace the manual numbered reference list with `whitepaper/references.bib` plus Pandoc citeproc.
- Preserve the full parsed source bibliography as a source registry or supplement, not as the main reference list.
- Use simple scalar author metadata so the PDF title page prints `David Ochabski`, not `true`.
- Remove the duplicate body H1 that repeats the document title.
- Remove manual leading numbers from Markdown headings when `numbersections: true` is enabled.
- Add unnumbered attributes to front matter, executive summary, appendices, and references.
- Run YAML/JSON/CFF validation already used by the repository.
- Render unified Markdown, HTML, and PDF if tooling exists.
- Confirm no L5 claims are introduced unless L5 release evidence is completed.
- Commit with a clear message: `Fix white paper citations and PDF heading build`.
- Tag only after final human edit and metadata freeze.

Appendix E: Glossary

Term	Working meaning
Artifact	Designed entity or thing created, instantiated, or specified in DSR.
Artifact universal	Reusable artifact kind or design specification distinct from a concrete instance.
Artifact instantiation	Concrete implementation, prototype, case, or technical object used to demonstrate or evaluate an artifact.
Boundary condition	Contextual condition limiting where a claim applies.
Contribution claim	Statement of what reusable or projectable knowledge has been added.
Design knowledge	Prescriptive or methodological knowledge connecting problem, intervention, mechanism, outcome, and boundary.
Demonstration	Showing artifact feasibility or use; not the same as evaluation unless criteria and evidence are defined.
Evaluation evidence	Evidence used to assess artifact, use, effect, or contribution claim.
Problem class	Abstracted class of problems beyond the local problem instance.
Problem instance	Situated local problem in a specific context.
Repository-native operationalization	Representing methodological commitments as structured files, metadata, records, schemas, examples, and release artifacts.
Traceability	Explicit link from problem to requirements, design decisions, artifact, evaluation, contribution, and reuse conditions.

References

Akoka, Jacky, Isabelle Comyn-Wattiau, Nicolas Prat, and Veda C. Storey. 2023. "Knowledge Contributions in Design Science Research: Paths of Knowledge Types." *Decision Support Systems* 166: 113898. https://doi.org/10.1016/j.dss.2022.113898.

Association for Computing Machinery. 2020. "Artifact Review and Badging Policy." https://www.acm.org/publications/policies/artifact-review-badging.

Baskerville, Richard, Abayomi Baiyere, Shirley Gregor, Alan Hevner, and Matti Rossi. 2018. "Design Science Research Contributions: Finding a Balance Between Artifact and Theory." *Journal of the Association for Information Systems* 19 (5): 358–76. https://doi.org/10.17705/1jais.00495.

Brendel, Alfred Benedikt, Tim-Benjamin Lembcke, Jan Muntermann, and Lutz M. Kolbe. 2021. "Toward Replication Study Types for Design Science Research." *Journal of Information Technology* 36 (3): 198–215. https://doi.org/10.1177/02683962211006429.

Dickhaut, Ernestine, Andreas Janson, and Jan Marco Leimeister. 2022. "Analyzing Design Knowledge Representation in Design Science Research and Deriving Recommendations to Support Design Knowledge Codification." In *Design Science Research: DESRIST 2022*, 417–28. Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-06516-3_31.

FAIR for Research Software Working Group. 2022. "FAIR Principles for Research Software (FAIR4RS Principles)." https://doi.org/10.15497/RDA00068.

Gregor, Shirley, and Alan R. Hevner. 2013. "Positioning and Presenting Design Science Research for Maximum Impact." *MIS Quarterly* 37 (2): 337–55. https://doi.org/10.25300/MISQ/2013/37.2.01.

Hevner, Alan R., Salvatore T. March, Jinsoo Park, and Sudha Ram. 2004. "Design Science in Information Systems Research." *MIS Quarterly* 28 (1): 75–105.

Hevner, Alan R., Jeffrey Parsons, Alfred Benedikt Brendel, Roman Lukyanenko, Verena Tiefenbeck, Monica Chiarini Tremblay, and Jan vom Brocke. 2024. "Transparency in Design Science Research." *Decision Support Systems* 182: 114236. https://doi.org/10.1016/j.dss.2024.114236.

Johannesson, Paul, and Erik Perjons. 2021. *An Introduction to Design Science*. 2nd ed. Springer. https://doi.org/10.1007/978-3-030-78132-3.

Maedche, Alexander, Shirley Gregor, Stefan Morana, and Jasper Feine. 2019. "Conceptualization of the Problem Space in Design Science Research." In *Design Science Research. Cases: DESRIST 2019*, 18–31. Springer. https://doi.org/10.1007/978-3-030-19504-5_2.

March, Salvatore T., and Gerald F. Smith. 1995. "Design and Natural Science Research on Information Technology." *Decision Support Systems* 15 (4): 251–66. https://doi.org/10.1016/0167-9236(94)00041-2.

Ochabski, David. 2026a. "Design Science Research Framework." https://doi.org/10.5281/zenodo.20241016.

———. 2026b. "Documentation Protocol." https://github.com/dochabski/documentation-protocol/releases/tag/v1.0.1.

Peffers, Ken, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. 2007. "A Design Science Research Methodology for Information Systems Research." *Journal of Management Information Systems* 24 (3): 45–77. https://doi.org/10.2753/MIS0742-1222240302.

Reining, Stefan, Frederik Ahlemann, Benjamin Mueller, and Rahul Thakurta. 2022. "Knowledge Accumulation in Design Science Research: Ways to Foster Scientific Progress." *The DATA BASE for Advances in Information Systems* 53 (1): 10–24.

Simon, Herbert A. 1996. *The Sciences of the Artificial*. 3rd ed. MIT Press.

Storey, Veda C., Richard L. Baskerville, and Mala Kaul. 2025. "Reliability in Design Science Research." *Information Systems Journal* 35 (3): 984–1014. https://doi.org/10.1111/isj.12564.

Venable, John, Jan Pries-Heje, and Richard Baskerville. 2016. "FEDS: A Framework for Evaluation in Design Science Research." *European Journal of Information Systems* 25 (1): 77–89. https://doi.org/10.1057/ejis.2014.36.

vom Brocke, Jan, Alan Hevner, and Alexander Maedche. 2020. "Introduction to Design Science Research." In *Design Science Research. Cases*, 1–13. Springer Nature Switzerland. https://doi.org/10.1007/978-3-030-46781-4_1.

vom Brocke, Jan, and Alexander Maedche. 2019. "The DSR Grid: Six Core Dimensions for Effectively Planning and Communicating Design Science Research Projects." *Electronic Markets* 29: 379–85. https://doi.org/10.1007/s12525-019-00358-7.

Weber, Ron. 2018. "Design-Science Research." In *Research Methods: Information, Systems, and Contexts*, 267–88. Elsevier. https://doi.org/10.1016/B978-0-08-102220-7.00011-X.

Weigand, Hans, Paul Johannesson, and Bo Andersson. 2021. "An Artifact Ontology for Design Science Research." *Data & Knowledge Engineering* 133: 101878. https://doi.org/10.1016/j.datak.2021.101878.